SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -1

CO4_AT1

Name : N.Sunil Kumar

Register No : 192472330

Course code : CSA0708

Course name: Computer Networks

Faculty name: Dr.S.Saraswathi

CODING CHALLENGE

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Answer :

Python Program

```
# Star Topology Generator
# Input: Number of hosts
n = int(input("Enter the number of hosts: "))
switch = "Switch-1"
print("\n========== STAR TOPOLOGY ==========\n")
print("          [", switch, "]")
print("               |")
# Generate topology
for i in range(1, n + 1):
    host = f"Host-{i}"
    cable_length = 5 * i   # Sample cable length in meters
    print(f"         |------ {host}")
print("\n========== CONNECTION DETAILS ==========\n")
for i in range(1, n + 1):
    host = f"Host-{i}"
   cable_length = 5 * i
    print(f"{host}  -->  {switch}  |  Cat6 UTP Cable Length: {cable_length} meters")
# Verification
print("\n========== VERIFICATION ==========")
print(f"Total Hosts Connected: {n}")
print(f"Central Device: {switch}")
```

```
if n > 0:
    print("Status: All hosts are successfully connected through the central switch.")
else:
    print("No hosts are connected.")
```

Expected output :-

```
========== STAR TOPOLOGY ==========
          [ Switch-1 ]
               |
         |------ Host-1
         |------ Host-2
         |------ Host-3
         |------ Host-4
========== CONNECTION DETAILS ==========
Host-1 --> Switch-1 | Cat6 UTP Cable Length: 5 meters
Host-2 --> Switch-1 | Cat6 UTP Cable Length: 10 meters
Host-3 --> Switch-1 | Cat6 UTP Cable Length: 15 meters
Host-4 --> Switch-1 | Cat6 UTP Cable Length: 20 meters
========== VERIFICATION ==========
Total Hosts Connected: 4
Central Device: Switch-1
Status: All hosts are successfully connected through the central switch.
```

2. Write a Python function named validate_segment(length_m) to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return True for valid cable lengths and return False along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Answer :-

```
# Function to validate Ethernet cable length
```

```
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print(f"Error: {length_m} meters exceeds the IEEE 802.3 maximum limit of 100 meters.")
        return False
# Test Cases
test_cases = [50, 100, 120]
print("===== Cable Length Validation =====")
for length in test_cases:
    result = validate_segment(length)
    print(f"Cable Length: {length} m --> {result}")
```

Expected output :-

```
===== Cable Length Validation =====
Cable Length: 50 m --> True
Cable Length: 100 m --> True
Error: 120 meters exceeds the IEEE 802.3 maximum limit of 100 meters.
Cable Length: 120 m --> False
```

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt,

and analyze how CSMA/CD minimizes data transmissionconflicts in Ethernet networks.

Answer :-

```
import random

hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]

log = open("csma_log.txt", "w")

log.write("CSMA/CD Simulation Log\n\n")

for i in range(1, 6):

    host = random.choice(hosts)

    print(f"Attempt {i}: {host} is transmitting...")

    log.write(f"Attempt {i}: {host} is transmitting...\n")

    if random.randint(1, 3) == 1:  # Collision chance

        backoff = random.randint(1, 5)

        print(f"  Collision! Back-off = {backoff} slots")

        print("  Retransmission successful.")

        log.write(f"Collision detected. Back-off: {backoff} slots\n")

        log.write("Retransmission successful.\n")

    else:

        print("  Transmission successful.")

        log.write("Transmission successful.\n")

log.close()

print("\nLog saved as csma_log.txt")
```

Expected output :-

```
Attempt 1: Host-2 is transmitting...
  Transmission successful.
```

Attempt 2: Host-1 is transmitting...

Collision! Back-off = 3 slots

Retransmission successful.

Attempt 3: Host-4 is transmitting...

Transmission successful.

Log saved as csma_log.txt

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

ANSWER:-

SERVER:-

```
import socket

mac_table = {}

server = socket.socket()

server.bind(("localhost", 5000))

server.listen(1)

print("Server waiting...")

conn, addr = server.accept()

data = conn.recv(1024).decode()

mac, port, msg = data.split(",")

mac_table[mac] = port

print("\nFrame Received:", msg)
```

```
print("\nMAC Address Table")
print("-------------------------")
print("MAC Address\tPort")
for m, p in mac_table.items():
    print(f"{m}\t{p}")
conn.send("Frame Forwarded".encode())
conn.close()
server.close()
```

CLIENT :-

```
import socket
client = socket.socket()
client.connect(("localhost", 5000))
mac = "AA:BB:CC:DD:EE:01"
port = "Port-1"
message = "Hello Server"
client.send(f"{mac},{port},{message}".encode())
print(client.recv(1024).decode())
client.close()
```

Expected output :-

```
Server waiting...
Frame Received: Hello Server
MAC Address Table
-------------------------
MAC Address          Port
AA:BB:CC:DD:EE:01    Port-1
```

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame.

Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

ANSWER :-

```
class StarSwitch:
    def __init__(self):
        self.ports = []
    def add_port(self, host_id):
        self.ports.append(host_id)
        print(f"{host_id} connected.")
    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(f"{host_id} disconnected.")
    def broadcast(self, source, frame):
        print(f"\nBroadcast from {source}: {frame}")
        for host in self.ports:
            if host != source:
                print(f"Frame received by {host}")
# Demo
witch = StarSwitch()
```

```
switch.add_port("Host-1")

switch.add_port("Host-2")

switch.add_port("Host-3")

switch.add_port("Host-4")

switch.broadcast("Host-1", "Hello Network")

switch.remove_port("Host-3")
```

Expected output :-

```
Host-1 connected.

Host-2 connected.

Host-3 connected.

Host-4 connected.

Broadcast from Host-1: Hello Network

Frame received by Host-2

Frame received by Host-3

Frame received by Host-4

Host-3 disconnected.
```